

```

import os
import httpx
import asyncio

from telegram import Update, ReplyKeyboardMarkup, KeyboardButton
from telegram.ext import Application, CommandHandler, MessageHandler, filters, Co

# 🗝️ ТОКЕНЫ — вставляются через переменные окружения на Railway
TELEGRAM_TOKEN = os.environ.get("TELEGRAM_TOKEN", "ВСТАВЬ_TELEGRAM_TOKEN")
ANTHROPIC_KEY = os.environ.get("ANTHROPIC_KEY", "ВСТАВЬ_ANTHROPIC_KEY")

# Состояния онбординга
STEP_FAMILY, STEP_BUDGET, STEP_CUISINE, STEP_ALLERGY = range(4)

# Тексты
TEXTS = {
    "welcome": "Салам! Мен Дарика 🌿\n\nЯ твой умный помощник на кухне!\nДавай по
    "step1": "👨‍👩‍👧‍👦 Кто у тебя в семье?",
    "step2": "💰 Бюджет на один ужин?",
    "step3": "🍳 Какую кухню любишь?",
    "step4": "🚫 Есть аллергии или запреты?\n\nНапиши (например: орехи, баранина)
    "ready": "Отлично! Теперь я всё знаю о твоей семье 🌿\n\nПросто напиши что ес
    "family_opts": ["Я одна", "Двое взрослых", "Семья с детьми", "Большая семья (
    "budget_opts": ["до 200 сом", "200-500 сом", "500-1000 сом", "без ограничений
    "cuisine_opts": ["Кыргызская 🇰🇬", "Узбекская 🇺🇿", "Казахская 🇰🇪", "Русская 🇷🇺
}

SYSTEM_PROMPT = """Ты — Дарика, тёплый ИИ-помощник по кулинарии для женщин Центра
Говори по-русски, тепло и по-домашнему, как подруга которая хорошо готовит.
Знаешь кыргызскую (манты, бешбармак, лагман, куурдак, шурпа, самса),
узбекскую, казахскую, русскую, европейскую и азиатскую кухни.

Профиль пользователя:
Семья: {family}
Бюджет: {budget}
Кухня: {cuisine}
Аллергии: {allergy}

Правила:
- Никогда не предлагай блюда с аллергенами
- Предлагай 2-3 конкретных блюда с кратким рецептом
- Учитывай бюджет и состав семьи
- Отвечай кратко, структурированно, с эмодзи
- Заканчивай вопросом или предложением"""

```

```

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Начало — приветствие и онбординг"""
    context.user_data.clear()
    await update.message.reply_text(TEXTS["welcome"])
    await asyncio.sleep(0.5)

    keyboard = [[KeyboardButton(opt)] for opt in TEXTS["family_opts"]]
    reply_markup = ReplyKeyboardMarkup(keyboard, resize_keyboard=True, one_time_k
    await update.message.reply_text(TEXTS["step1"], reply_markup=reply_markup)
    return STEP_FAMILY

async def step_family(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data["family"] = update.message.text
    keyboard = [[KeyboardButton(opt)] for opt in TEXTS["budget_opts"]]
    reply_markup = ReplyKeyboardMarkup(keyboard, resize_keyboard=True, one_time_k
    await update.message.reply_text(TEXTS["step2"], reply_markup=reply_markup)
    return STEP_BUDGET

async def step_budget(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data["budget"] = update.message.text
    keyboard = [[KeyboardButton(opt)] for opt in TEXTS["cuisine_opts"]]
    reply_markup = ReplyKeyboardMarkup(keyboard, resize_keyboard=True, one_time_k
    await update.message.reply_text(TEXTS["step3"], reply_markup=reply_markup)
    return STEP_CUISINE

async def step_cuisine(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data["cuisine"] = update.message.text
    await update.message.reply_text(TEXTS["step4"])
    return STEP_ALLERGY

async def step_allergy(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    context.user_data["allergy"] = "нет" if text in ["/нет", "нет", "Нет"] else t

    # Показываем главное меню
    keyboard = [
        ["🍲 Что приготовить?", "📅 17 План на неделю"],
        ["🥗 ПП рецепт", "⚡ Быстрый ужин"],
        ["🍝 Кыргызская кухня", "🛒 Список покупок"],
        ["⚙️ Мой профиль", "🔄 Начать заново"],
    ]

    reply_markup = ReplyKeyboardMarkup(keyboard, resize_keyboard=True)
    await update.message.reply_text(TEXTS["ready"], reply_markup=reply_markup)
    return ConversationHandler.END

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Обработка всех сообщений после онбординга"""

```

```

user_text = update.message.text

# Кнопка профиля
if user_text == "👤 Мой профиль":
    profile = context.user_data
    if not profile.get("family"):
        await update.message.reply_text("Сначала пройди настройку! Напиши /start")
        return
    msg = f"👤 Твой профиль:\n\n👨‍👩‍👧 Семья: {profile.get('family', '-')} \n💰 Бю:"
    await update.message.reply_text(msg)
    return

# Кнопка начать заново
if user_text == "🔄 Начать заново":
    context.user_data.clear()
    await start(update, context)
    return

# Проверяем что онбординг пройден
if not context.user_data.get("family"):
    await update.message.reply_text("Привет! Нажми /start чтобы начать 🌱")
    return

# Показываем что печатаем
await update.message.chat.send_action("typing")

# Формируем системный промпт с профилем
system = SYSTEM_PROMPT.format(
    family=context.user_data.get("family", "не указано"),
    budget=context.user_data.get("budget", "не указано"),
    cuisine=context.user_data.get("cuisine", "не указано"),
    allergy=context.user_data.get("allergy", "нет"),
)

# История сообщений
if "history" not in context.user_data:
    context.user_data["history"] = []

context.user_data["history"].append({"role": "user", "content": user_text})

# Оставляем только последние 10 сообщений
if len(context.user_data["history"]) > 10:
    context.user_data["history"] = context.user_data["history"][-10:]

# Запрос к Anthropic
try:
    async with httpx.AsyncClient(timeout=30) as client:

```

```

        response = await client.post(
            "https://api.anthropic.com/v1/messages",
            headers={
                "Content-Type": "application/json",
                "x-api-key": ANTHROPIC_KEY,
                "anthropic-version": "2023-06-01",
            },
            json={
                "model": "claude-haiku-4-5-20251001",
                "max_tokens": 1000,
                "system": system,
                "messages": context.user_data["history"],
            }
        )

    data = response.json()
    reply = "".join(b.get("text", "") for b in data.get("content", []))

    if not reply:
        reply = "Упс, что-то пошло не так 🤔 Попробуй ещё раз!"

    context.user_data["history"].append({"role": "assistant", "content": repl
    await update.message.reply_text(reply)

except Exception as e:
    await update.message.reply_text("Ошибка соединения 🙏 Попробуй через мину

def main():
    app = Application.builder().token(TELEGRAM_TOKEN).build()

    # Онбординг
    conv = ConversationHandler(
        entry_points=[CommandHandler("start", start)],
        states={
            STEP_FAMILY: [MessageHandler(filters.TEXT & ~filters.COMMAND, step_f
            STEP_BUDGET: [MessageHandler(filters.TEXT & ~filters.COMMAND, step_b
            STEP_CUISINE: [MessageHandler(filters.TEXT & ~filters.COMMAND, step_c
            STEP_ALLERGY: [MessageHandler(filters.TEXT & ~filters.COMMAND, step_a
        },
        fallbacks=[CommandHandler("start", start)],
    )

    app.add_handler(conv)
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_messag

    print("Дарика бот запущен! 🌱")
    app.run_polling(drop_pending_updates=True)

```

```
if __name__ == "__main__":  
    main()
```